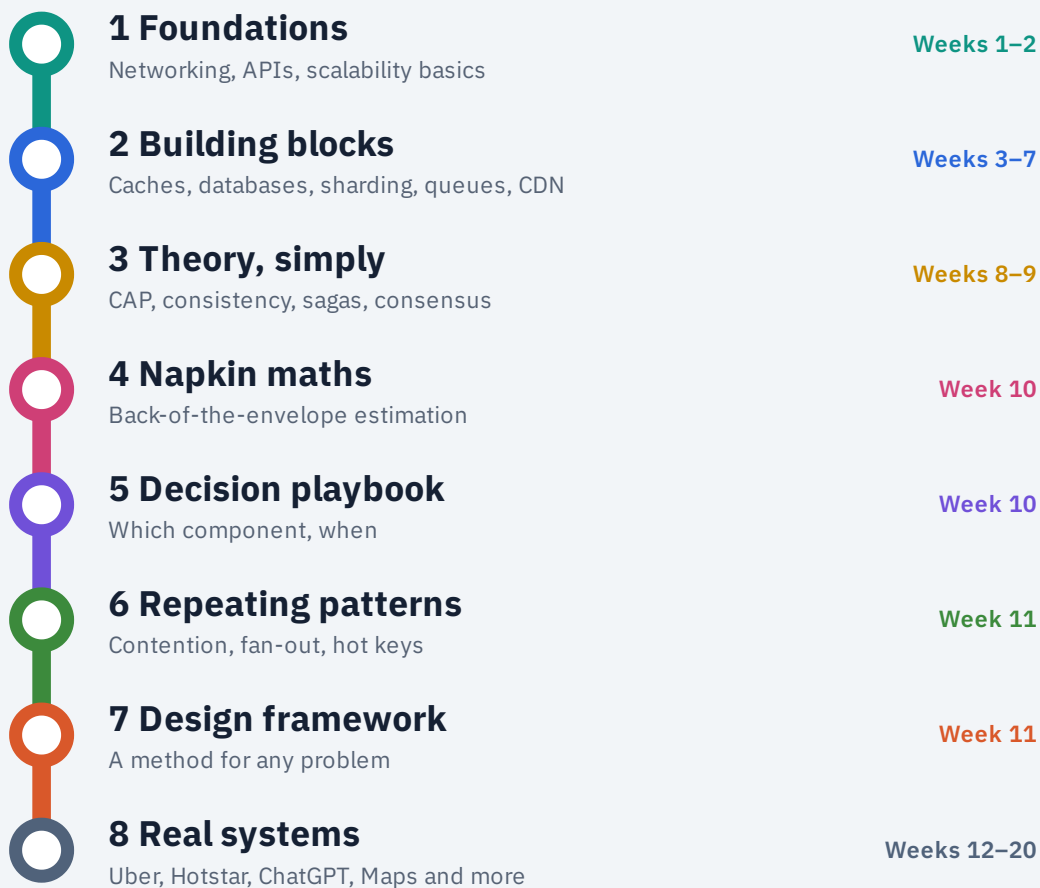


System design, from your first server to Uber

A beginner-first roadmap for high-level design. Learn each building block, learn when to pick it, learn to estimate with napkin maths, then take apart the systems you use every day. Tick off each station with a pen as you finish it.



A beginner-first high-level design roadmap: 47 stations, 21 real systems

Contents

0	How to use this roadmap	2
1	Foundations: how the internet actually talks	3
	What system design is (and isn't), Networking essentials, APIs and client-server basics, The qualities every design is judged on	
2	Building blocks: the Lego pieces of every system	5
	Load balancers, Caching, CDN (Content Delivery Network), Databases: SQL and the NoSQL family, Database replication, Partitioning and sharding, Consistent hashing, Message queues and event streams, Real-time communication, Storage: blocks, files and objects, Rate limiting, Unique ID generation, Search and indexing, Data structures for scale, Coordination and high availability, Microservices plumbing and resilience	
3	Theory, simply: consistency, CAP and friends	10
	CAP and PACELC, the easy version, Consistency models and quorums, Distributed transactions, Consensus, time and ordering, Architecture styles, Big data: batch and stream processing, Operating it: observability, deployment, security	
4	Napkin maths: back-of-the-envelope estimation	12
	The numbers to memorise, The five-step recipe, Worked examples	
5	The decision playbook: which component, when	15
	SQL or NoSQL?, Queue, Kafka or pub/sub?, Polling, SSE, WebSockets or webhooks?, Do I need a cache? Which strategy?, Synchronous or asynchronous?, More quick decisions, How many servers?	
6	Patterns that repeat across every design	19
	Scaling reads, Scaling writes, Real-time updates and fan-out, Dealing with contention, Multi-step processes, Handling large blobs, Long-running tasks, Traffic spikes and hot keys, Proximity search	
7	A framework for designing anything	21
	The delivery framework	
8	Real systems: take apart the apps you use	23
	Starter tier: one core idea each, Intermediate tier: several blocks working together, Advanced tier: the famous ones	
+	Famous system design blogs to follow	33
	Newsletters and explainers, Global engineering blogs, Indian engineering blogs, Must-read posts and papers	

How to use this roadmap

System design feels huge because it mixes networking, databases, distributed systems and product thinking. The trick is order. Every big design (YouTube, Uber, ChatGPT) is just 10–15 building blocks wired together with a handful of trade-offs. So you first learn the blocks, then the rules for choosing between them, then practise wiring them together on real products.

This roadmap blends the structure of AlgoMaster's fundamentals course (concept by concept, from networking to security), Hello Interview (a delivery framework, key technologies and repeating patterns), Design Gurus' Grokking (explicit trade-off lessons and classic case studies) and Educative's building-block approach, plus the standard books and engineering blogs listed at the end.

Budget about an hour a day. For every station: read one explanation, draw it on paper from memory, then answer the "Decide" question out loud. If you can explain a concept to a friend in two minutes with a diagram, tick it off. Phases 1–3 build vocabulary, phases 4–7 build judgment, and phase 8 is where it all turns into skill. Don't wait until you "finish" theory before attempting designs; start the starter designs in phase 8 around week 6.

1 Foundations: how the internet actually talks

Before choosing components you need the vocabulary: what a request is, how it travels, and what "scalable" or "available" really mean.

Weeks 1–2

1 What system design is (and isn't)

High-level design (HLD) decides the boxes and arrows: services, databases, caches, queues and how data flows between them. Low-level design (LLD) decides classes and code inside one box. HLD is about trade-offs, not a single right answer.

HLD vs LLD

Functional requirements (what it does)

Non-functional requirements (how well: latency, availability, scale, consistency, cost)

Monolith as a valid starting point

Reading an architecture diagram

Decide: For a food delivery app, list 4 functional and 4 non-functional requirements. Which non-functional one would hurt the business most if it failed at 8 pm on a Saturday?

2 Networking essentials

Every design starts with "the client sends a request". Know what happens between typing a URL and seeing a page, because latency, security and load balancing all live here.

IP addresses, ports

DNS resolution and TTLs

TCP vs UDP

HTTP/1.1, HTTP/2, HTTP/3 (QUIC)

HTTPS and TLS handshake

OSI model (just layers 3, 4, 7)

Forward proxy vs reverse proxy

Checksums

Decide: TCP when every byte must arrive in order (APIs, payments, chat). UDP when speed beats completeness and a lost packet is fine (live video calls, online games, DNS lookups).

3 APIs and client-server basics

The API is the contract of your system. A good one is predictable, paginated, versioned and safe to retry.

REST resources and verbs

Status codes that matter (200, 201, 202, 400, 401, 403, 404, 409, 429, 500, 503)

Pagination: offset vs cursor

Idempotency and idempotency keys

GraphQL

gRPC and Protobuf

JSON vs binary formats

Versioning

AuthN vs AuthZ, sessions vs JWT, OAuth2, SSO

Decide: REST for public and mobile APIs (simple, cacheable). gRPC between internal services that call each other thousands of times a second (fast, typed, streaming). GraphQL when many different clients need different shapes of the same data and you want to avoid over-fetching.

4 The qualities every design is judged on

These words appear in every interview and design review. Learn to attach a number to each one.

Scalability: vertical vs horizontal

Stateless vs stateful servers

Availability and "nines"

Reliability and fault tolerance

Single point of failure (SPOF)

Latency vs throughput vs bandwidth

p50, p95, p99 latency

Redundancy: active-active vs active-passive

Regions and availability zones

Decide: Scale vertically (a bigger machine) until it's expensive or risky, because it's simple. Scale horizontally (more machines) once one box can't cope or can't be allowed to fail. Horizontal scaling only works easily for stateless servers, which is why you push state into databases and caches.

2 Building blocks: the Lego pieces of every system

The core of the roadmap. Each block below shows up in almost every famous design. For each one, learn what it is, how it fails, and when to reach for it.

Weeks 3–7

5 Load balancers

Spread traffic across many servers and route around dead ones. The first component you add after a single server.

L4 (TCP) vs L7 (HTTP) balancing Round robin, weighted, least connections, IP hash Health checks
Sticky sessions (and why to avoid them) DNS load balancing, GeoDNS Anycast
Nginx, HAProxy, AWS ALB/NLB, Envoy Avoiding the LB as a SPOF

Decide: L7 when you need to route by URL path, headers or cookies (for example `/api/payments` to one service). L4 when you need raw speed or non-HTTP traffic such as millions of WebSocket connections. Use least-connections when requests vary a lot in duration.

6 Caching

The single most powerful performance tool. RAM is roughly 1,000 times faster than a database round trip, so serving hot data from memory changes everything.

Where to cache: browser, CDN, app memory, distributed cache Cache-aside, read-through
Write-through, write-back, write-around Eviction: LRU, LFU, TTL Invalidation strategies
Cache stampede / thundering herd Hot keys Cache warming Redis vs Memcached

Decide: Default to cache-aside with a TTL. Add a cache when reads outnumber writes (10:1 or more), data can be a few seconds stale, and the same items are requested again and again. Don't cache bank balances you're about to debit; do cache product pages, user profiles and trending lists.

7 CDN (Content Delivery Network)

A worldwide network of caches near your users. A viewer in Chennai gets the video segment from a server in Chennai, not from Virginia.

Edge locations and points of presence Pull vs push CDN Cache keys and TTLs Invalidation / purge
Origin shielding Signed URLs Video segments via CDN Multi-CDN

Decide: Anything that is the same for many users belongs on a CDN: images, JS/CSS, video, and even API responses like a live cricket score JSON with a 1–2 second TTL.

8 Databases: SQL and the NoSQL family

Choosing the database is the most consequential decision in most designs. Learn the families by their access pattern, not by brand name.

Relational model and joins ACID transactions Isolation levels (read committed, repeatable read, serializable)
Indexes: B-tree / B+ tree LSM trees (why Cassandra writes fast) Write-ahead log and durability Key-value stores
Document stores Wide-column stores Graph databases Time-series databases Search engines
Vector databases Normalization vs denormalization

Decide: Start with PostgreSQL or MySQL unless you have a concrete reason not to. The full SQL vs NoSQL decision table is in [phase 5](#).

9 Database replication

Keep copies of the same data on several machines, so one failure doesn't lose data and reads can be spread out.

Leader-follower (primary-replica) Multi-leader Leaderless (Dynamo-style)
Synchronous vs asynchronous vs semi-sync Replication lag Read-your-own-writes problem Failover and promotion
Split brain

Decide: Replicate when you need high availability or more read capacity. Replication does *not* help write capacity, because every copy still has to apply every write. If writes are the bottleneck, you need sharding.

Example: after a user updates their profile, send their own reads to the leader for a few seconds so they never see the old version from a lagging replica.

10 Partitioning and sharding

Split one huge dataset across many machines, each owning a slice. This is how you scale writes and storage past a single server.

Vertical vs horizontal partitioning Shard key selection Range-based sharding Hash-based sharding
Directory / lookup-based sharding Geo sharding Hot shards and celebrity keys Cross-shard queries and joins
Resharding / rebalancing Secondary indexes on sharded data Vitess, Citus

Decide: Pick the shard key from your most common query. Chat messages: shard by `conversation_id` so a chat's history sits on one shard. Orders: shard by `user_id` for "my orders". Avoid keys that concentrate traffic, like timestamp (all new writes hit one shard) or country (India would melt its shard).

11 Consistent hashing

With plain `hash(key) % N`, adding one server moves almost every key. Consistent hashing places servers and keys on a ring so adding or removing a server moves only about $1/N$ of the keys.

The hash ring Virtual nodes for even spread Replication along the ring
Used in DynamoDB, Cassandra, CDNs, distributed caches Rendezvous hashing (alternative)

Decide: Use it whenever nodes join and leave often (cache clusters, storage nodes) and moving data is expensive.

12 Message queues and event streams

Let one part of the system hand work to another without waiting. Queues absorb spikes, decouple services, and make retries possible.

Point-to-point queue vs pub/sub

Queue (RabbitMQ, SQS) vs log (Kafka, Pulsar, Kinesis)

Kafka topics, partitions, offsets, consumer groups

At-most-once, at-least-once, exactly-once

Ordering guarantees

Dead-letter queues

Backpressure

Change data capture (CDC, Debezium)

Transactional outbox

Decide: A queue is a to-do list: each job is done once by one worker, then gone. Kafka is a newspaper archive: many different readers can read the same events at their own pace and re-read old ones. Full table in [phase 5](#).

13 Real-time communication

How the server pushes updates to users: chat messages, driver locations, live scores, AI answers streaming word by word.

Short polling

Long polling

Server-Sent Events (SSE)

WebSockets

Webhooks

WebRTC

Connection gateways and a user→server registry

Heartbeats and reconnection

Decide: Updates flow one way (server to client)? SSE. Both ways, frequently, with low latency? WebSockets. Server-to-server notification? Webhook. Full table in [phase 5](#).

14 Storage: blocks, files and objects

Photos, videos, PDFs and backups don't belong in your database. They go to object storage, and the database stores only a pointer.

Block vs file vs object storage

S3 / GCS / Azure Blob

Pre-signed URLs

Multipart and resumable uploads

Chunking and content hashing

Distributed file systems (GFS, HDFS)

Replication vs erasure coding

Storage tiers (hot, cold, archive)

Decide: Anything larger than a few KB that you read whole goes to object storage. Let the client upload directly using a pre-signed URL, so gigabytes never pass through your app servers.

15 Rate limiting

Protect your system from abuse, bugs and traffic spikes by capping how many requests a client can make.

Token bucket

Leaky bucket

Fixed window counter

Sliding window log

Sliding window counter

Distributed limiting with Redis + Lua

Where to limit: gateway, service, per user, per IP, per API key

HTTP 429 and Retry-After

Load shedding

Decide: Token bucket is the default: it allows short bursts but enforces an average rate. For an AI chat product, rate-limit by tokens consumed, not just requests.

16 Unique ID generation

Once data is spread across many machines, a single auto-increment counter becomes a bottleneck. You need IDs that are unique, fast, and often sortable by time.

Database auto-increment Multi-master with step offsets UUID v4 (random) UUID v7 / ULID (time-ordered)
Ticket server (Flickr) Twitter Snowflake: timestamp + machine ID + sequence Clock skew issues

Decide: Need IDs roughly sorted by time (feeds, messages)? Snowflake or UUIDv7. Just need uniqueness? UUIDv4. Need short, human-friendly codes (URL shortener)? A counter or pre-generated key range, encoded in base62.

17 Search and indexing

Databases are bad at "find restaurants whose name or dishes contain 'biryani', typo-tolerant, near me". Search engines are built for exactly that.

Inverted index Tokenization, stemming Relevance scoring (TF-IDF, BM25) Elasticsearch / OpenSearch
Keeping search in sync via CDC Tries for autocomplete Top-k suggestions Semantic search with embeddings

Decide: Keep your database as the source of truth and feed a search index from it asynchronously. Search can be a few seconds behind; that's usually fine.

18 Data structures for scale

Clever structures that trade a tiny error rate for huge savings in memory, or that make location queries fast.

Bloom filter ("definitely not" / "maybe yes") Cuckoo filter HyperLogLog (count unique viewers in ~12 KB)
Count-min sketch (approximate top-k) Geohash Quadtree Google S2, Uber H3 Skip lists (Redis sorted sets)
Merkle trees (sync and anti-entropy)

Decide: Bloom filter to skip expensive lookups (has this crawler seen this URL? is this username taken?). HyperLogLog for "unique visitors". Geohash or H3 for "drivers near me".

19 Coordination and high availability

When many machines work together, someone must decide who's the leader, who's alive, and who holds a lock.

Heartbeats and failure detection Leader election ZooKeeper, etcd Distributed locks and fencing tokens
Gossip protocol Redis replication Redis Sentinel (monitoring + automatic failover)
Redis Cluster (sharding over 16,384 hash slots) Split brain and quorums

Decide: One Redis node with replicas and you want automatic failover? Redis Sentinel. Data or traffic too big for one Redis node? Redis Cluster, which shards and fails over. Need a trustworthy leader or config store? etcd or ZooKeeper, built on consensus.

20 Microservices plumbing and resilience

Once you split a system into services, network calls fail in new ways. These patterns keep one slow service from taking down everything.

API gateway

Backend for Frontend (BFF)

Service discovery

Timeouts

Retries with exponential backoff and jitter

Circuit breaker

Bulkhead

Sidecar and service mesh

Graceful degradation

Decide: Every network call gets a timeout. Every retry needs backoff plus jitter, and the operation must be idempotent. If a dependency keeps failing, a circuit breaker stops calling it and serves a fallback, like hiding recommendations instead of failing the whole page.

3 Theory, simply: consistency, CAP and friends

Now that you've seen replication and sharding, the theory has something to attach to. Keep it practical: every idea here answers "what does the user see when things go wrong?"

Weeks 8–9

21 CAP and PACELC, the easy version

Networks between data centres will sometimes break (a partition). When that happens, a replicated system must choose: refuse some requests to stay correct (consistency), or keep answering with possibly stale data (availability). You can't have both during the partition. PACELC adds the everyday part: even with no failures, you trade latency against consistency.

C, A, P in plain words

Why "pick 2 of 3" is misleading (P isn't optional)

CP examples: bank ledger, seat booking, etcd

AP examples: likes, feeds, DNS, shopping cart

PACELC

Choosing per feature, not per system

Decide: Ask "what's worse: showing wrong data, or showing an error?" Wrong balance or a double-booked seat is worse than an error, so choose consistency. A like count off by 3 is better than an error, so choose availability. One app can make both choices in different places: BookMyShow is CP for seat locking and AP for movie reviews.

22 Consistency models and quorums

"Consistency" is a spectrum. Knowing the levels lets you promise exactly what users need and nothing more expensive.

Strong / linearizable

Sequential and causal

Read-your-writes

Monotonic reads

Eventual consistency

Quorum: N replicas, W writes, R reads; $R + W > N$

Tunable consistency in Cassandra and DynamoDB

Conflict resolution: last-write-wins, vector clocks, CRDTs

Decide: With $N = 3$, $W = 2$, $R = 2$, every read overlaps at least one replica holding the latest write. Lower W or R for speed when staleness is acceptable.

23 Distributed transactions

Placing a food order touches orders, payments, restaurant and delivery services. How do you keep them consistent when there's no single database transaction?

Why distributed transactions are hard

Two-phase commit (2PC) and its blocking problem

Saga: choreography vs orchestration

Compensating actions (refund, release seat)

Transactional outbox

Idempotent consumers

Workflow engines (Temporal, AWS Step Functions)

Decide: Inside one database, use a normal transaction. Across services, use a saga: a sequence of local steps, each with an "undo". Use the outbox pattern so "save to DB" and "publish event" never get out of sync.

24 Consensus, time and ordering

Learn this at "can explain in two minutes" depth. It's what makes etcd, ZooKeeper, Kafka's controller and Spanner work.

Why clocks can't be trusted

Lamport timestamps

Vector clocks

Raft: leader, terms, log replication, majority

Paxos (awareness only)

Google Spanner and TrueTime (awareness)

Decide: Never build consensus yourself; use etcd or ZooKeeper. Remember that majority-based systems need an odd number of nodes (3 or 5) and tolerate $(N - 1) / 2$ failures.

25 Architecture styles

The overall shape of a system. Most big companies started as monoliths and split later.

Monolith and modular monolith

Microservices

Event-driven architecture

CQRS (separate write and read models)

Event sourcing

Serverless

Peer-to-peer

Hexagonal architecture (awareness)

Decide: Small team, new product: modular monolith. Many teams that need to deploy independently, or parts that scale very differently (video transcoding vs user profiles): microservices.

26 Big data: batch and stream processing

Analytics, recommendations, fraud detection, trending topics and live leaderboards all run on data pipelines.

Batch vs stream

MapReduce idea

Spark

Flink and Kafka Streams

Windowing: tumbling, sliding, session

ETL / ELT

Data lake, warehouse, lakehouse

Lambda vs Kappa architecture

OLTP vs OLAP (Postgres vs ClickHouse / BigQuery)

Decide: Result needed in seconds (surge pricing, fraud checks, live viewer counts)? Stream processing. Result can wait hours (daily reports, model training, monthly settlements)? Batch.

27 Operating it: observability, deployment, security

A design isn't finished until you can see it, ship it safely and secure it. Mentioning these in a design review signals real-world maturity.

Logs, metrics, traces

Correlation IDs

SLI, SLO, SLA, error budgets

Alerting

Rolling, blue-green, canary deployments

Feature flags

Schema migrations without downtime

TLS, encryption at rest

Secrets management

RBAC

Multi-region and disaster recovery (RPO, RTO)

4 Napkin maths: back-of-the-envelope estimation

Estimation isn't about precision. It answers design questions: do we need a cache? Do we need sharding? 5 servers or 500? You only need rounding, powers of ten, and about fifteen numbers.

Week 10

28 The numbers to memorise

Round aggressively. A day has 86,400 seconds; call it 100,000 (10^5). That one trick makes most maths mental.

Shortcut	Value	Why it matters
Seconds in a day	$\approx 10^5$	Divide daily counts by 100k to get average QPS
1 million per day	$\approx 12 / \text{s}$	Tiny. One server handles it
100 million per day	$\approx 1,200 / \text{s}$	A few servers
1 billion per day	$\approx 12,000 / \text{s}$	Needs a fleet, caching, probably sharding
Peak vs average	$\times 2$ to $\times 5$	Plan capacity for peak; live events can spike $\times 10$ or more
Thousand, million, billion, trillion	KB, MB, GB, TB	10^3 steps; 1 million $\times$ 1 KB = 1 GB
Days in a year	≈ 400	Rounding 365 up keeps maths easy and adds headroom

Size of things	Rough size
ID / timestamp (long)	8 B
UUID	16 B
Typical DB row (user, order, message with metadata)	0.5–1 KB
Compressed photo	200 KB – 2 MB
1 minute of 1080p video (compressed)	≈ 30 –50 MB
1 minute of audio	≈ 1 MB
LLM token	≈ 4 characters of English

Latency ladder (orders of magnitude)	Time
Read from RAM	~ 100 ns
Redis GET in the same data centre (incl. network)	~ 0.5 –1 ms
SSD random read	~ 0.1 ms
Simple indexed DB query	~ 1 –10 ms
Round trip within one region	~ 1 –2 ms
Round trip India $\leftrightarrow$ US	~ 200 –250 ms

Human "feels instant"

< 100–200 ms

What one machine can roughly do	Ballpark	Note
Stateless app server, simple JSON API	1k–10k req/s	Use ~1k–2k if each request does real work
PostgreSQL / MySQL node	~5k–20k simple writes/s; more reads	Depends heavily on hardware and indexes
Redis node	~100k+ ops/s	Single-threaded command execution
Cassandra node	~10k–50k writes/s	Scales roughly linearly with nodes
Kafka broker	~100s of MB/s	Millions of small messages/s across a cluster
WebSocket gateway	~50k–500k idle connections	Use 100k in estimates
Single DB comfortable size	~1–5 TB	Beyond that, think sharding or archiving

Availability	Downtime per year	Per month
99% (two nines)	≈ 3.65 days	≈ 7.3 h
99.9%	≈ 8.8 h	≈ 44 min
99.99%	≈ 53 min	≈ 4.4 min
99.999%	≈ 5.3 min	≈ 26 s

These are interview-grade ballparks, not benchmarks. Real numbers depend on hardware, payload size and query shape; what matters is getting the order of magnitude right.

29 The five-step recipe

- 1 **Users.** Start from daily active users (DAU). If given monthly users, DAU is often 20–50% of MAU.
- 2 **Traffic.** Writes/day = DAU × writes per user. Reads/day = DAU × reads per user. Divide each by 10^5 for average QPS, then multiply by 2–5 for peak. The read:write ratio tells you whether you need caching and replicas.
- 3 **Storage.** Writes/day × size per write = daily storage. × 400 = yearly. × years kept × 3 replicas = total. Estimate media separately from metadata; it's usually 100× bigger.
- 4 **Bandwidth.** QPS × payload size. Big egress numbers mean "put it on a CDN".
- 5 **Servers.** Peak QPS ÷ what one server can do × 1.5 headroom, then spread across at least 2–3 availability zones. For connections: concurrent users ÷ connections per gateway.

Only estimate what changes the design. If reads are 200k/s, the conclusion is "one database can't serve this, we need a cache and replicas". Say the conclusion out loud; that's the point of the maths.

30 Worked examples

A Twitter-like feed. 200M DAU; each user posts 2 times and views 100 posts per day.

Step	Maths	Result and conclusion
Write QPS	$200M \times 2 / 10^5$	$\approx 4k/s$, peak $\approx 12k/s$. More than one SQL node likes: shard, or use Cassandra
Read QPS	$200M \times 100 / 10^5$	$\approx 200k/s$, peak $\approx 600k/s$. Read:write is 50:1, so cache heavily and precompute feeds
Text storage	$400M \times 1 \text{ KB}$	400 GB/day $\approx 160 \text{ TB/year}$; $\times 5 \text{ years} \times 3 \text{ replicas} \approx 2.4 \text{ PB}$. Sharding is mandatory
Media	10% with 500 KB image	$40M \times 500 \text{ KB} = 20 \text{ TB/day}$. Object storage plus CDN
App servers	$600k \div 5k \times 1.5$	≈ 180 servers across 3 zones

WhatsApp-style chat connections. 100M users online at peak, each holding one WebSocket. $100M \div 100k$ per gateway $\approx 1,000$ gateway servers, plus a registry that maps user $\rightarrow$ gateway so messages can be routed.

Live cricket score for 30M viewers. If every app polls every 5 seconds, that's $30M \div 5 = 6M$ requests/s. Terrifying for your servers, trivial for a CDN: the score JSON is identical for everyone, so edges cache it for $\sim 1\text{--}2 \text{ s}$ and your origin sees only a few requests per second per edge. The napkin maths tells you the architecture.

5 The decision playbook: which component, when

This is the part most beginners are missing. Each table turns a vague "it depends" into concrete signals you can look for in the requirements.

Week 10, then keep it open while practising

31 SQL or NoSQL?

Signal in the requirements	Lean towards
Money, inventory, bookings, anything that must never double-count	SQL (ACID transactions)
Data has relationships and you'll query it many ways (users, orders, restaurants, menus)	SQL
Access pattern still unclear, early product	SQL; it's the most flexible
Enormous write volume with simple access by key (chat messages, events, IoT, location history)	Wide-column: Cassandra, ScyllaDB, Bigtable
Simple get/put by key at huge scale with predictable latency (sessions, carts, user settings)	Key-value: DynamoDB, Redis
Flexible, nested, varying schema read as one blob (product catalog, CMS content)	Document: MongoDB, Firestore
Multi-hop relationship queries (friends of friends, fraud rings)	Graph: Neo4j, Neptune
Metrics and time-stamped measurements	Time-series: Prometheus, InfluxDB, TimescaleDB
Full-text, fuzzy or faceted search	Elasticsearch / OpenSearch, alongside your main DB
"Find similar meaning" for AI and recommendations	Vector: pgvector, Pinecone, Milvus
Analytics over billions of rows	Columnar warehouse: ClickHouse, BigQuery, Snowflake
Files, images, video	Object storage (S3, GCS); metadata in a DB

Rule of thumb: real systems are polyglot. Zomato-style apps plausibly use SQL for orders and payments, Redis for sessions and live state, Elasticsearch for search, and Kafka to connect them.

32 Queue, Kafka or pub/sub?

Need	Pick	Example
Distribute jobs to workers; each job done once, then deleted; per-message retries and DLQ	Queue: SQS, RabbitMQ, Celery / Sidekiq on Redis	Send emails, resize images, transcode videos

High-throughput event stream read by many independent consumers, with replay and ordering per key	Log: Kafka, Kinesis, Pulsar	Order events feed billing, analytics, notifications and search indexing
Fan out one message to many subscribers right now, loss acceptable	Pub/sub: Redis Pub/Sub, SNS, Google Pub/Sub	Push a chat message to whichever gateway servers hold the recipients
Complex multi-step business process with retries, timeouts and human steps	Workflow engine: Temporal, Step Functions	Payment → restaurant accept → rider assignment

Rule of thumb: "Do this task" → queue. "This happened" and several teams care → Kafka.

33 Polling, SSE, WebSockets or webhooks?

Technique	Direction	Use when	Real example
Short polling	Client asks repeatedly	Updates are infrequent, a few seconds' delay is fine, or the response can be CDN-cached	Checking a report's status; live score JSON behind a CDN
Long polling	Client asks, server holds until news	Fallback when you can't keep persistent connections	Older chat systems, some sync clients
SSE	Server → client only	Streaming one-way updates over plain HTTP with automatic reconnect	ChatGPT / Claude streaming tokens; notifications; live commentary
WebSockets	Both ways, persistent	Frequent two-way messages with low latency	WhatsApp chat, multiplayer games, Google Docs cursors, Uber driver app
Webhooks	Server → another server	Tell a third party when something finishes	Razorpay or Stripe telling a merchant a payment succeeded
WebRTC	Peer to peer media	Audio and video calls	Google Meet, WhatsApp calls

34 Do I need a cache? Which strategy?

Situation	Answer
Read-heavy (10:1 or more), same items requested often, slight staleness OK	Yes: cache-aside + TTL
Expensive computation reused (feed, recommendations, search results)	Yes: cache the computed result
Users must see their own write immediately	Write-through, or delete the cache key on write
Extremely write-heavy counters (views, likes)	Write-back: count in Redis, flush to DB in batches (accept small loss risk)
Long-tail access, every key requested once	Probably not; the hit rate will be low
Strictly correct values (balance before debit, seat availability at checkout)	Read from the source of truth; cache only for display

One key gets millions of hits (celebrity post, match score)

Replicate the hot key, add a local in-process cache, or push to CDN

35 Synchronous or asynchronous?

Go synchronous (request / response) when

The user needs the result to continue (login, fetch menu)

It finishes in well under a second

The chain of calls is short (2–3 hops)

Failure should be shown immediately

Go asynchronous (queue + worker) when

The work is slow: seconds to hours (video transcoding, report generation, ML jobs)

Traffic is spiky and you want to smooth it (flash sale orders)

Many downstream systems react to one event

A third party is slow or flaky and retries are needed

The async recipe: accept the request → store a job → return 202 Accepted with a job ID → a worker processes it from a queue → notify the client by push, SSE, webhook or polling the job status.

36 More quick decisions

Question	Pick A when...	Pick B when...
Replication or sharding?	Replicate: need HA or more read capacity	Shard: writes or data size exceed one node
Strong or eventual consistency?	Strong: money, inventory, bookings, unique usernames	Eventual: likes, views, feeds, search, recommendations
Push or pull for feeds?	Push (fan-out on write): normal users with few followers	Pull (fan-out on read): celebrities with millions of followers; most systems do a hybrid
Load balancer or API gateway?	LB: spread traffic across identical servers	Gateway: auth, rate limiting, routing to many services; usually you have both
Monolith or microservices?	Monolith: small team, early product	Microservices: many teams, very different scaling needs
Redis Sentinel or Redis Cluster?	Sentinel: data fits one node, need automatic failover	Cluster: need to shard data across nodes
CDN or direct from server?	CDN: static or shared content, global users, big files	Direct: personalised, private, rapidly changing responses
Optimistic or pessimistic locking?	Optimistic (version check): conflicts are rare	Pessimistic / reservation hold: many users fight for the same item (seats, flash sale)

37 How many servers?

The formula: $\text{servers} = \text{peak QPS} \div \text{QPS per server} \times \text{headroom (1.3–2)}$, then round up so every availability zone has at least 2. Never plan for 100% utilisation; aim for 50–70% so a zone can fail and the others absorb its traffic.

Peak load	Rough fleet (at ~2k req/s per server)
100 req/s	2–3 servers (just for redundancy)
5,000 req/s	≈ 4–5 servers
50,000 req/s	≈ 40–50 servers, plus caching becomes essential
500,000 req/s	Hundreds of servers; most reads must hit cache or CDN

"5 or 15 servers" is rarely the interesting question. The interesting one is which layer breaks first: usually the database, then hot keys, then connection limits.

6 Patterns that repeat across every design

Hello Interview and AlgoMaster both organise their advanced material around these. Learn each once and you'll recognise it inside a dozen different products.

Week 11

38 Scaling reads

Indexes → caching → read replicas → CDN → denormalised or precomputed views. Apply in that order; each step is more complex than the last.

Seen in: Instagram feed, product pages, YouTube video metadata.

39 Scaling writes

Batch writes → write-optimised DB (LSM) → shard by a well-spread key → buffer through Kafka → aggregate in memory before writing (count 1,000 likes, write once).

Seen in: Uber location updates, ad click aggregation, view counters, metrics.

40 Real-time updates and fan-out

Persistent connection gateways, a registry of who is connected where, pub/sub between services and gateways, and push vs pull fan-out for large audiences.

Seen in: WhatsApp, live comments, live scores, Uber tracking, notifications.

41 Dealing with contention

Many users want the same thing at once. Tools: DB row locks, optimistic concurrency with version numbers, atomic Redis operations, distributed locks with TTL, and "hold for 10 minutes" reservations with virtual waiting rooms.

Seen in: BookMyShow seats, Tatkal train tickets, flash sales, online auctions.

42 Multi-step processes

Sagas, state machines and workflow engines with idempotent steps and compensations.

Seen in: food order lifecycle, payments, ride lifecycle, onboarding flows.

43 Handling large blobs

Pre-signed URLs, chunked resumable uploads, content hashing for dedupe, CDN for downloads, processing pipelines after upload.

Seen in: Google Drive, YouTube uploads, WhatsApp media.



44 Long-running tasks

Job queue, workers, status table, progress updates, retries, timeouts, dead-letter queues.

Seen in: video transcoding, report exports, LLM agent tasks, web crawling.

45 Traffic spikes and hot keys

Pre-scaling before known events, autoscaling, queue-based load levelling, rate limiting, load shedding, graceful degradation, and splitting or replicating hot keys.

Seen in: India vs Pakistan on JioHotstar, Big Billion Days sale, New Year's Eve messages and food orders.

46 Proximity search

Geohash, quadtree or H3 cells to find "things near me" without scanning everything; Redis GEO for moving objects, Elasticsearch or PostGIS for static ones.

Seen in: Uber, Zomato, Google Maps nearby places, Tinder.

7 A framework for designing anything

Use this same sequence for every practice problem, every interview, and every real design doc. Timings are for a 45–60 minute interview.

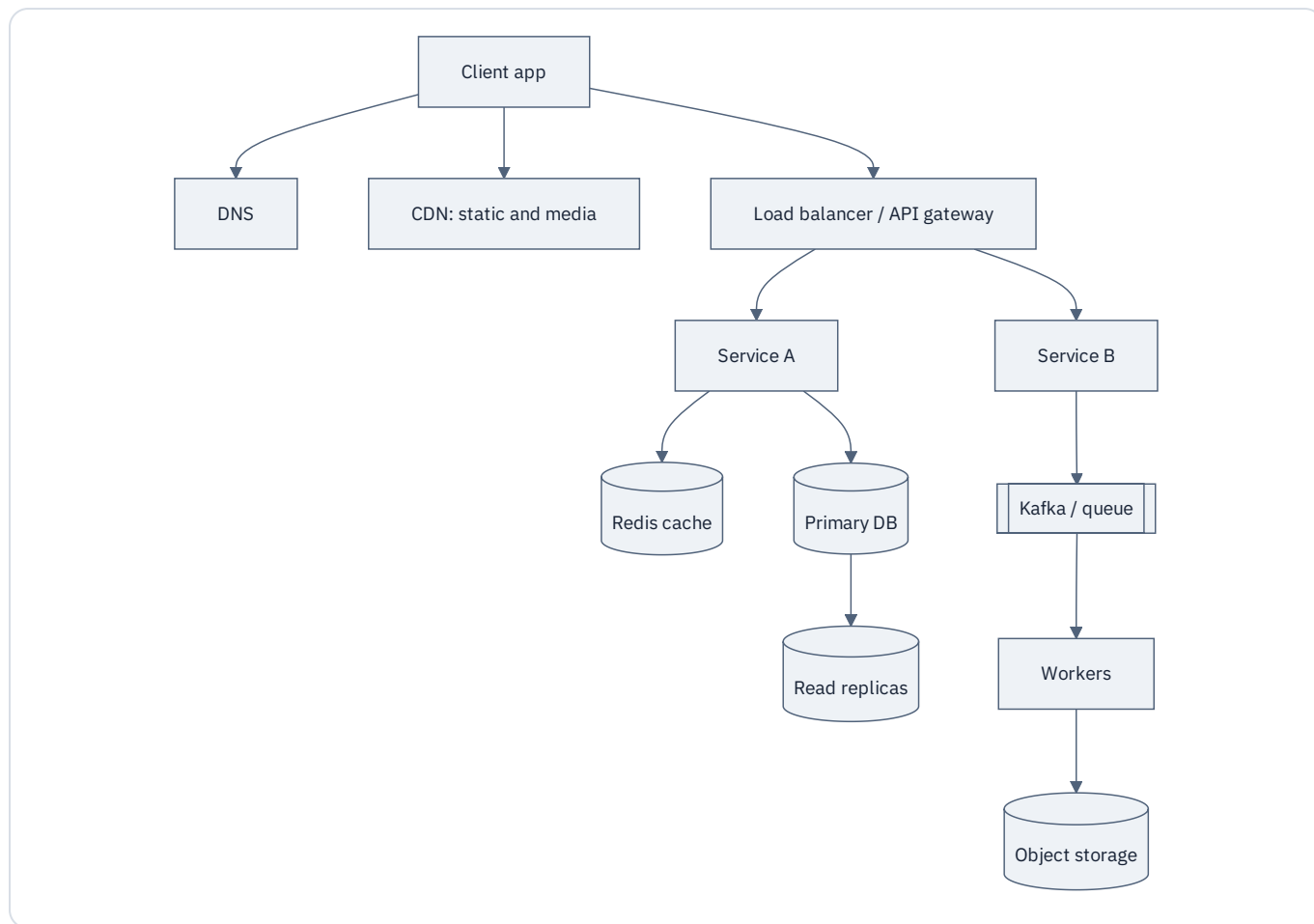
Week 11



47 The delivery framework

- 1 **Requirements (≈5 min).** 3–5 core functional requirements ("users can book a ride"), explicitly mark what's out of scope, then non-functional ones with numbers: scale, latency target, availability, consistency needs.
- 2 **Estimate (≈3–5 min).** Only the numbers that shape the design: QPS, read:write ratio, storage, concurrent connections.
- 3 **Core entities and API (≈5 min).** The main nouns (User, Ride, Driver, Location) and the endpoints or events between client and server.
- 4 **High-level design (≈10–15 min).** Draw the simplest design that satisfies every functional requirement: client → LB / gateway → services → databases. Walk one request end to end.
- 5 **Deep dives (≈15–20 min).** Go back to the non-functional requirements and fix what breaks: hot spots, bottlenecks, SPOFs, consistency and failure scenarios. This is where seniority shows.
- 6 **Wrap up.** State the trade-offs you made, what you'd monitor, and what you'd do with more time.

Common beginner mistakes: jumping to Kafka and microservices before the simple design works; estimating for 10 minutes without drawing conclusions; naming technologies without saying why; never discussing what happens when a component dies.



The skeleton almost every design grows from. Your job in each case study is to decide which boxes to add, remove or scale, and why.

8 Real systems: take apart the apps you use

Work through these in order; each tier reuses what the previous one taught. For each problem: design it yourself on paper first (40 minutes, using the framework), then compare with a course write-up or the company's engineering blog.

Starter tier from week 6; the rest weeks 12–20

Starter tier: one core idea each

URL shortener (Bitly / TinyURL)

IDs, base62, caching, read-heavy

The hard part

Generating short unique codes at scale and serving redirects in milliseconds. Reads outnumber writes about 100:1.

Building blocks

- Counter or pre-allocated ID ranges encoded in base62 (7 characters $\approx$ 3.5 trillion codes)
- Key-value store for code $\rightarrow$ URL
- Redis cache for hot links; 301 vs 302 redirects (analytics)
- Async click analytics via Kafka

 I designed this

Rate limiter

token bucket, Redis, gateway

The hard part

Counting accurately across many servers without adding latency to every request.

Building blocks

- Token bucket or sliding window counter
- Redis with atomic Lua scripts
- Rules per user, IP, API key, endpoint
- Fail-open vs fail-closed when Redis is down

 I designed this

Unique ID generator and key-value store

Snowflake, consistent hashing, quorum

The hard part

Two classic "build the building block" questions. The KV store (think Amazon's Dynamo paper) combines almost everything from phase 2 and 3.

Building blocks

- Snowflake layout: 41-bit timestamp, 10-bit machine, 12-bit sequence
- Consistent hashing with virtual nodes
- Replication and N/R/W quorums
- Gossip for membership, Merkle trees for repair, hinted handoff

 I designed this

Pastebin and notification service

object storage, queues, retries

The hard part

Pastebin teaches metadata vs blob separation. Notifications teach fan-out to email, SMS and push with retries, user preferences, templates and deduplication.

 I designed this

Intermediate tier: several blocks working together

WhatsApp / chat

WebSockets, registry, delivery states

The hard part

Delivering messages instantly to users connected to any of a thousand gateway servers, and reliably to users who are offline.

Building blocks

- WebSocket gateways behind L4 load balancers
- Session registry (Redis): user → gateway
- Message store sharded by conversation (Cassandra-style) with time-ordered IDs
- Sent / delivered / read acknowledgements
- Group fan-out; push notifications for offline users
- Media via object storage + CDN; end-to-end encryption (awareness)

 I designed this

Instagram / Twitter news feed

fan-out, celebrity problem, caching

The hard part

Building a personalised feed for hundreds of millions of users fast, when some accounts have 100 million followers.

Building blocks

- Fan-out on write into per-user feed caches (Redis lists)
- Fan-out on read for celebrities; hybrid merge
- Post storage sharded, media on CDN
- Ranking service; cursor pagination

 I designed this

YouTube / Netflix

upload pipeline, transcoding, adaptive streaming, CDN

The hard part

Accepting huge uploads, converting each into many resolutions, and streaming smoothly worldwide on any network.

Building blocks

- Resumable chunked upload to object storage with pre-signed URLs
- Transcoding as a DAG of jobs on a queue with many workers
- HLS / DASH: video split into few-second segments at several bitrates; the player switches quality as bandwidth changes
- CDN serves segments (Netflix runs its own, Open Connect)
- Metadata in sharded MySQL (YouTube created Vitess for this)
- View counts via async aggregation

☐ I designed this

Google Drive / Dropbox

chunking, dedupe, sync, metadata vs blobs

The hard part

Syncing files across devices efficiently: re-uploading only changed parts, handling offline edits and conflicts, and sharing securely.

Building blocks

- Files split into blocks (a few MB each), identified by content hash, so identical blocks are stored once and only changed blocks are uploaded
- Metadata DB (SQL): files, folders, versions, block lists, permissions
- Block storage in object storage
- Sync service notifies other devices (long polling or WebSocket), which pull the delta
- Conflict copies, version history, ACLs for sharing

☐ I designed this

Typeahead / search autocomplete

tries, top-k, caching

The hard part

Returning suggestions within ~50 ms for every keystroke.

Building blocks

- Trie with precomputed top-k per prefix, held in memory
- Offline batch job rebuilds rankings from query logs
- Client debouncing, browser and CDN caching for short prefixes

☐ I designed this

BookMyShow / Ticketmaster

contention, seat holds, waiting rooms

The hard part

Thousands of people clicking the same seat in the same second when a blockbuster opens, without double-booking.

Building blocks

- SQL with row-level locks or conditional updates for seats
- Temporary holds with TTL (Redis) while the user pays
- Virtual waiting room / queue to meter traffic
- Read-heavy browsing cached; Elasticsearch for event search

☐ I designed this

Zomato / Swiggy food delivery

geo search, order state machine, dispatch

The hard part

Three-sided marketplace (customer, restaurant, rider) with a huge dinner-time peak, and an order that moves through many states across services.

Building blocks

- Restaurant discovery: Elasticsearch with geo filters, serviceability zones, heavy caching of menus
- Cart and order service in SQL; order as a state machine (placed → accepted → preparing → picked up → delivered)
- Payments via a PSP with idempotency keys
- Kafka events drive restaurant tablet notifications, rider dispatch, customer updates
- Rider assignment: geo index of available riders, batching orders, ETA models
- Live tracking via WebSockets or SSE

☐ I designed this

Web crawler

URL frontier, politeness, Bloom filters

The hard part

Downloading billions of pages without hammering any single site or fetching the same page twice.

Building blocks

- URL frontier with per-domain queues and rate limits (robots.txt)
- Bloom filter for seen URLs; content hashing / SimHash for near-duplicates
- Distributed fetchers, DNS cache, object storage for pages

☐ I designed this

Advanced tier: the famous ones

Uber / Ola ride hailing

location firehose, geo index, matching

The hard part

Every active driver sends GPS every few seconds. With 1M active drivers updating every 4 s, that's 250k writes/s: far too many for a normal database, and the data is stale within seconds anyway.

Building blocks

- Driver app → WebSocket gateways → location service
- Latest locations kept in memory in a geo index (Uber built and open-sourced H3, a hexagonal grid); history streamed to Kafka for analytics
- Matching: search nearby cells, rank by ETA, offer to one driver at a time with a timeout, lock the driver to avoid double assignment
- Trip service in a strongly consistent store; trip as a state machine
- Surge pricing from stream processing over supply and demand per cell
- ETA and routing from a maps service

○ I designed this

Google Maps

tiles, graph routing, live traffic

The hard part

Showing a map of the whole planet smoothly, finding the fastest route across millions of road segments in milliseconds, and knowing where traffic is right now.

Building blocks

- Map tiles: the world pre-cut into squares at each zoom level (increasingly vector tiles), served from CDN
- Road network as a graph partitioned by region; routing with A* plus precomputation such as contraction hierarchies so long routes stay fast
- Live traffic from aggregated, anonymised phone location streams, processed in real time to update edge weights
- ETA prediction with ML on historical and live speeds
- Places and geocoding via geospatial and text search indexes
- Navigation: client follows the route, reroutes on deviation or new traffic

○ I designed this

Google Search

crawl, inverted index, scatter-gather

The hard part

Searching an index of hundreds of billions of pages and returning ranked results in a fraction of a second.

Building blocks

- Crawler (above) feeding a processing pipeline: parse, dedupe, extract links, compute link-based signals like PageRank
- Inverted index (word → list of documents) partitioned by document across thousands of shards, each replicated
- Query path: parse and spell-correct → scatter the query to all shards → each returns its local top results → gather, merge and re-rank with ML → snippets
- Caching of popular query results; autocomplete via tries
- Historical foundations worth reading: the GFS, MapReduce and Bigtable papers

○ I designed this

The hard part

Tens of millions of people watching one match at the same moment, all joining when a star walks in to bat, and the score, viewer count and reactions updating live for all of them. Indian streaming platforms have publicly reported concurrency records in the tens of millions during big cricket matches.

Video path

- Stadium feed → encoders produce multiple bitrates → packager cuts HLS/DASH segments of a few seconds
- Segments pushed to multiple CDNs close to viewers; clients use adaptive bitrate so a 4G user in a train still gets a picture
- Ads stitched in at breaks (server-side ad insertion)

Live score path

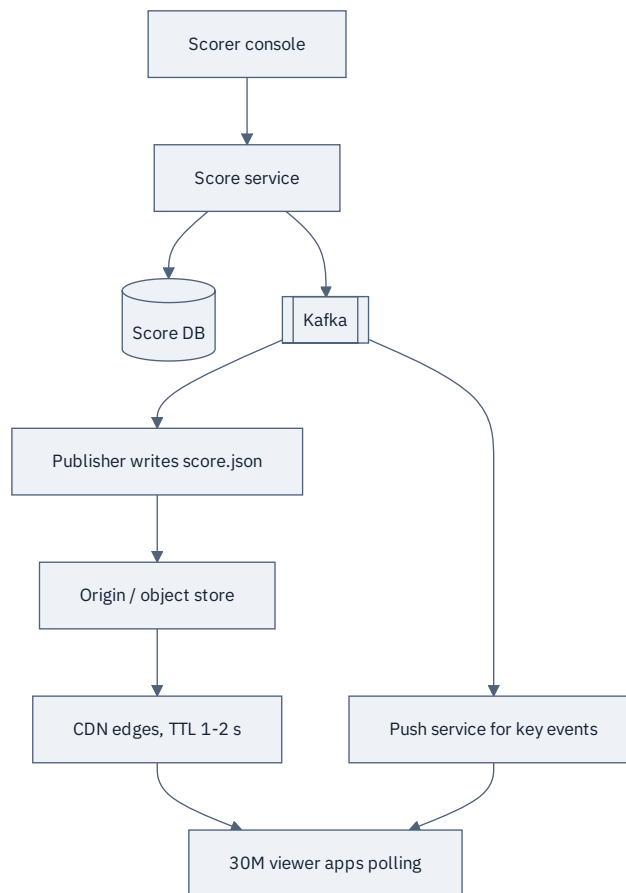
- Scorers enter each ball → score service writes to its DB and publishes an event
- The latest score is written as a small JSON document and served through the CDN with a TTL of about 1–2 seconds
- Clients poll that URL every few seconds (or get it via SSE / a push channel). Because the score is identical for every viewer, the CDN absorbs millions of requests per second and the origin barely notices. This is the napkin maths from station 30 in action
- Key moments (wicket, six) can also trigger a push to update the UI instantly

Viewer count and reactions

- Clients send heartbeats; counts are aggregated per region in stream processors and merged, then shown approximately ("2.1 Cr watching")
- Emoji reactions are sampled and aggregated through Kafka rather than delivered one by one

Surviving the spike

- Pre-scale before the toss based on predicted concurrency; reactive autoscaling is too slow when millions join in a minute
- Graceful degradation: turn off non-essential features (recommendations, comments) under load so video keeps playing
- Rate limiting and caching on login, entitlement and home-page APIs, which spike when a match starts



○ I designed this

Payment system (UPI, Razorpay, Stripe)

idempotency, ledgers, reconciliation

The hard part

Money must never be lost or charged twice, even when networks time out halfway through and banks respond minutes later.

Building blocks

- Idempotency keys on every payment API so retries don't double-charge
- Payment as a state machine: created → pending → authorised → captured → settled, plus failed / refunded
- Double-entry ledger in a strongly consistent SQL database; append-only, every debit has a matching credit
- External calls to banks and networks (for UPI: payer PSP → NPCI → banks) with timeouts; a timeout means "unknown", resolved later by status checks
- Outbox pattern for events; webhooks to merchants with retries and signatures
- Daily reconciliation batch jobs matching internal ledger against bank settlement files
- Fraud and risk checks, card tokenisation and PCI DSS compliance (awareness)

○ I designed this

The hard part

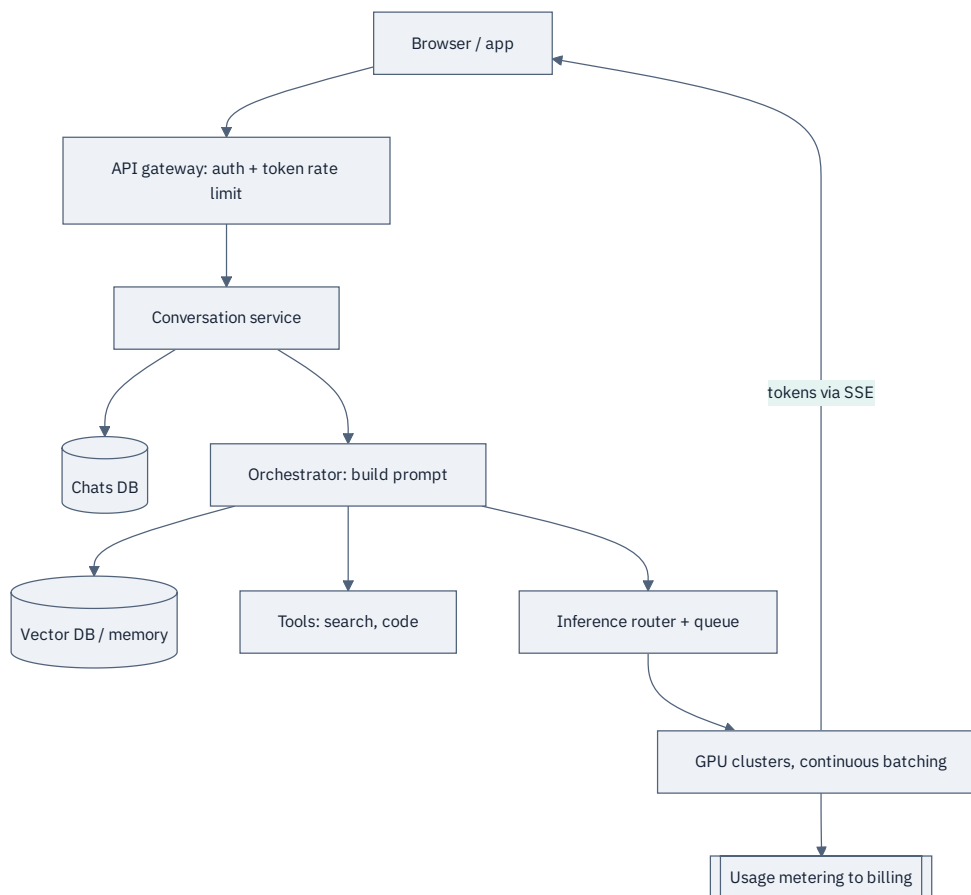
Each request is expensive (it occupies GPU capacity for seconds), responses must stream word by word, GPUs are the scarce resource, and demand is spiky. This is a queueing and capacity problem more than a database problem.

Building blocks

- API gateway: authentication, and rate limits measured in tokens per minute and requests per minute per user or plan
- Conversation service stores chats and messages (sharded by user or conversation); attachments in object storage
- Orchestrator builds the prompt: system prompt, conversation history truncated or summarised to fit the context window, retrieved documents or memory from a vector DB, tool results
- Inference router picks a model and a GPU cluster, with a queue in front for fairness and priority by plan
- Inference servers use continuous batching (many users' requests share a GPU), a KV cache for attention state, and prompt caching for repeated prefixes
- Tokens stream back to the browser via Server-Sent Events; both OpenAI's and Anthropic's public APIs stream this way
- Safety classifiers on input and output; tools (web search, code execution) as separate sandboxed services
- Usage metering pipeline (Kafka → billing) and heavy observability on latency to first token and tokens per second

Napkin maths to try

If one GPU server produces ~1,000–2,000 output tokens/s with batching, and at peak 1M users each want 50 tokens/s streamed, you need $50\text{M tokens/s} \div \sim 1,500 \approx 33,000$ GPU servers' worth of capacity. Numbers like this explain why rate limits, queues and smaller routed models exist. (Illustrative figures; real throughput varies hugely by model and hardware.)



○ I designed this

Google Docs: collaborative editing

OT / CRDTs, WebSockets

The hard part

Several people typing in the same paragraph at once and everyone ending with the same document.

Building blocks

- WebSocket session per document, routed so a document's editors reach the same server
- Operational Transformation (Google Docs) or CRDTs to merge concurrent edits
- Operation log plus periodic snapshots; presence and cursors via pub/sub

○ I designed this

Top-K, leaderboards and ad-click aggregation

stream processing, count-min sketch, Redis sorted sets

The hard part

Counting billions of events in near real time (top videos, trending hashtags, ad spend, fantasy cricket leaderboards like Dream11) with accuracy good enough to bill on.

Building blocks

- Kafka ingest → Flink windowed aggregation → OLAP store (ClickHouse, Druid)
- Redis sorted sets for leaderboards; count-min sketch for approximate heavy hitters
- Batch reconciliation for exact billing numbers

○ I designed this

Also worth doing

stock exchange, job scheduler, metrics monitoring, Airbnb, Spotify, e-commerce flash sale

Once the list above feels comfortable, these add new twists: an in-memory matching engine and strict ordering (stock exchange / Zerodha), distributed cron with leases (job scheduler), time-series ingestion and alerting (metrics), availability calendars and double-booking (Airbnb), audio streaming and offline downloads (Spotify), and inventory contention at extreme spikes (Flipkart Big Billion Days).

○ I designed these



Famous system design blogs to follow

Courses teach the building blocks; engineering blogs show how real companies used them, including what broke. Read one post a week while you work through phase 8, and try to redraw the architecture from memory afterwards.

Newsletters and explainers

Blog	Why read it	Link
ByteByteGo (Alex Xu)	Visual, bite-sized explanations of concepts and real architectures; the author of <i>System Design Interview</i>	blog.bytebytego.com
AlgoMaster Newsletter (Ashish Pratap Singh)	Beginner-friendly concept deep dives and interview insights	blog.algomaster.io
High Scalability	The classic archive of "how X scaled" architecture write-ups	highscalability.com
The Pragmatic Engineer (Gergely Orosz)	How big tech actually builds and runs systems, with inside detail	newsletter.pragmaticengineer.com
Martin Fowler	The reference for architecture patterns: microservices, CQRS, event sourcing, sagas	martinfowler.com
All Things Distributed (Werner Vogels, Amazon CTO)	Distributed systems thinking from the team behind Dynamo and S3	allthingsdistributed.com
AWS Architecture Blog	Reference architectures and well-architected patterns	aws.amazon.com/blogs/architecture

Global engineering blogs

Company	Best known for	Link
Netflix TechBlog	Video streaming at scale, chaos engineering, microservices resilience	netflixtechblog.com
Uber Engineering	Geospatial indexing (H3), real-time matching, huge data platforms	uber.com/blog/engineering
Meta Engineering	Caching at planet scale, social graph storage, infrastructure	engineering.fb.com
Discord	Storing trillions of chat messages, real-time at scale	discord.com/blog
Cloudflare	Networking, CDN, DDoS protection and edge computing, explained deeply	blog.cloudflare.com
Slack Engineering	Real-time messaging, database scaling with Vitess	slack.engineering
Stripe	Payments reliability, idempotent API design	stripe.com/blog/engineering
Dropbox Tech	File sync engines, storage infrastructure	dropbox.tech

Airbnb Engineering	Search, payments, service-oriented architecture migration	medium.com/airbnb-engineering
LinkedIn Engineering	Kafka was born here; feeds and data infrastructure	linkedin.com/blog/engineering
Pinterest Engineering	Feeds, sharding MySQL, recommendation systems	medium.com/pinterest-engineering
Shopify Engineering	Surviving flash sales like Black Friday, modular monoliths	shopify.engineering
Google Research	The source of GFS, MapReduce, Bigtable and Spanner ideas	research.google/blog

Indian engineering blogs

Company	Best known for	Link
Hotstar / JioHotstar	Live cricket for tens of millions of concurrent viewers, handling "tsunami" traffic spikes	blog.hotstar.com
Swiggy Bytes	Food and quick-commerce delivery, dispatch, ETAs, peak-hour scaling	bytes.swiggy.com
Zomato	Food delivery platform, and handling New Year's Eve order peaks	blog.zomato.com
Razorpay Engineering	Payment systems, UPI, reliability and idempotency	engineering.razorpay.com
Zerodha Tech	Stock broking at scale with a small team; pragmatic, boring-tech choices	zerodha.tech
Flipkart Tech	Big Billion Days sale traffic, e-commerce inventory	blog.flipkart.tech
PhonePe Tech	UPI payments at massive transaction volumes	tech.phonepe.com

Must-read posts and papers

Search these titles; each one maps directly to a station in this roadmap.

Title	Source	Teaches
Scaling Is Not An Accident; T for Tsunami: Dealing with Traffic Spikes	Hotstar blog	Pre-scaling instead of autoscaling for live cricket spikes (station 45, JioHotstar case)
How Discord Stores Trillions of Messages	Discord blog	Chat storage, Cassandra to ScyllaDB migration, hot partitions (stations 10, 12)
H3: Uber's Hexagonal Hierarchical Spatial Index	Uber Engineering	Geospatial indexing for "drivers near me" (stations 18, 46)
Sharding & IDs at Instagram	Instagram Engineering	Sharding Postgres and time-sortable IDs (stations 10, 16)
Announcing Snowflake	Twitter Engineering	Distributed unique ID generation (station 16)
Designing Robust and Predictable APIs with Idempotency	Stripe blog	Idempotency keys and safe retries (stations 3, 20, payments case)

How Figma's Multiplayer Technology Works	Figma blog	Real-time collaboration and conflict resolution (Google Docs case)
Rewriting the Heart of Our Sync Engine	Dropbox Tech	File sync design (Google Drive case)
Scaling Datastores at Slack with Vitess	Slack Engineering	Sharding MySQL in production (station 10)
Dynamo: Amazon's Highly Available Key-Value Store	Amazon (paper, 2007)	Consistent hashing, quorums, eventual consistency (stations 11, 22)
Scaling Memcache at Facebook	Meta (NSDI 2013 paper)	Distributed caching, invalidation, thundering herds (station 6)
The Google File System; MapReduce; Bigtable	Google (papers)	Distributed storage and batch processing foundations (stations 14, 26)
Kafka: a Distributed Messaging System for Log Processing	LinkedIn (paper, 2011)	Why a log beats a queue for event streams (station 12)

How to read an engineering post: before reading the solution, write down how you would solve their problem. Then read, and note three things: the constraint that forced their choice, the trade-off they accepted, and what they would do differently. That habit turns reading into design skill.